

7.3 🚗 Car Simulation

🎯 Objective

By the end of this exercise, you will know how to:

- Translate and rotate the canvas to draw more dynamic animations.
- Manipulate shapes on the canvas with keystrokes.
- Simulate entities on the canvas having a velocity using vectors and trigonometry.

🔨 Setup

1. Create a new folder in your `420-426-Multimedia-and-Internet` repo called `7.3-Car-Simulation` using the `Finder`.
2. Inside `7.3-Car-Simulation`, create an `index.html`, `car.js`, and `main.js` and link them together as usual. Make sure you declare `car.js` before `main.js` since `main` needs `car` to be declared before using it.
3. Open `7.3-Car-Simulation` inside `VS Code`.
4. Open `index.html` in Chrome.

As you complete each problem/step, make sure to **commit as you go!** This will keep track of all your changes as you code. What you do **not** want to do is complete the exercise without committing, and do one giant commit at the end.

Main

Here's some starter code for `main.js`:

```
let canvas = document.querySelector('canvas');
let context = canvas.getContext('2d');

canvas.height = 500;
canvas.width = 500;

car = new Car(canvas.width / 2, canvas.height / 2);

function animate() {
  requestAnimationFrame(animate);
  context.clearRect(0, 0, canvas.width, canvas.height);
```

```

    // Check which key was pressed and call the appropriate Car function.

    car.update();
}

animate();

canvas.focus();

```

Before you copy-paste, I want you to go line-by-line and explain to yourself what each line of code is doing. If you get stuck, refer to **exercise 7.2**. If you're still unsure after that, **Google it**. If you're still unsure after that, **ask me** 😊

Representing the Car

We want all of our car properties and methods to be *encapsulated* so that they are in a nice, neat, reusable package. To do so, declare a **Car** class in **car.js** with:

- **Properties**

- **x** and **y** that are initialized based on the constructor parameters.
- **width = 50**
- **height = 25**
- **speed = 0**
 - The velocity the car is currently going.
- **maxSpeed = 5**
 - The velocity the car cannot go above.
- **rotation = 0**
 - The angle (in **radians**) the car is positioned.
- **friction = 0.95**
 - The rate at which the car will decelerate when no user input is being received.

- **Functions**

- **constructor(x, y)**
- **draw()**
 - All of the **context** draw methods will be called here.
 - Gets called in the **update** method.
- **update()**
 - Change the cars position on the canvas based on its current **speed** and **rotation**.
 - Call this method on each frame of our **animation**.

- `accelerate()`
 - Increase the car's speed.
- `decelerate()`
 - Decrease the car's speed.
- `reverse()`
 - Increase the car's speed in the opposite direction.
- `turnRight()`
 - Rotate the car clockwise.
- `turnLeft()`
 - Rotate the car counterclockwise.
- `displayStatus()`
 - `Position` , `speed` , and `rotation` will be written on the screen to help with debugging as we go.

Here's the `displayStatus()` function:

```
context.save();
context.font = "12px Arial";
context.fillText(`Position: (${this.position.x.toFixed(0)},
${this.position.y.toFixed(0)})`, 10, 20);
context.fillText(`Rotation: ${this.rotation.toFixed(2)}°`, 10, 40);
context.fillText(`Speed: ${this.speed.toFixed(2)}`, 10, 60);
context.restore();
```

Drawing the Car

We've already seen how to draw shapes on the canvas in [exercise 7.2](#) . For this exercise, we will represent our car with a simple rectangle. Here's an example `draw()` method:

```
context.save();
context.translate(this.position.x, this.position.y);
context.rotate(this.rotation * Math.PI / 180);
context.fillStyle = "MediumPurple";
context.fillRect(0, 0, this.width, this.height);
context.restore();
```

Hold on, it seems like there are some methods here that we haven't seen before!

Canvas Transformations

The content of this section is taken from the [Mozilla Developer Docs](#) 🙏

Until now, we only used the default grid and changed the size of the overall canvas for our needs. With transformations there are more powerful ways to translate the origin to a different position, rotate the grid and even scale it.

Saving and Restoring State

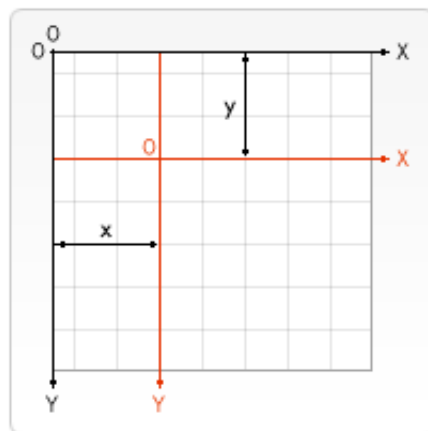
Before we look at the transformation methods, let's look at two other methods which are indispensable once you start generating ever more complex drawings.

- `save()` : Saves the entire state of the canvas.
- `restore()` : Restores the most recently saved canvas state.

It's a good idea to `save` the canvas state before doing any transformations. In most cases, it is just easier to call the `restore` method than having to do the equivalent reverse transformation to return to the original state.

Translation

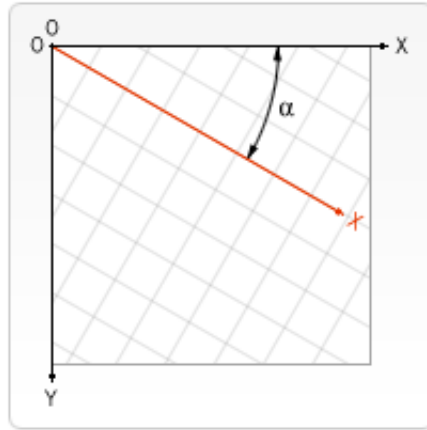
The first of the transformation methods we'll look at is `translate()` . This method is used to move the canvas and its origin to a different point in the grid.



- `translate(x, y)` : Moves the canvas and its origin on the grid. `x` indicates the horizontal distance to move, and `y` indicates how far to move the grid vertically.

Rotation

The second transformation method is `rotate()` . We use it to rotate the canvas around the current origin.



- `rotate(angle)` : Rotates the canvas clockwise around the current origin by the angle number of radians.

The rotation center point is always the canvas origin. To change the center point, we will need to move the canvas by using the `translate()` method.

Using states, translations, and rotations enables us to draw our car anywhere on the canvas and in any direction we want!

Driving the Car

The car will react based on keyboard input from the user. Namely, the `keydown` event.

1. Keep track of which key was pressed by adding an event listener to the canvas when the user presses an arrow key.
2. Inside of the `animate` function, check if any of the four arrow keys was pressed. If so, `console.log` a message to ensure that the event listener is working:

```
↑ was pressed!  
↓ was pressed!  
← was pressed!  
→ was pressed!
```

Accelerating

Let's start by moving the car towards the right side of the canvas when the user hits accelerate:

1. Call `car.accelerate()` when the user hits the `up` arrow key.
2. Inside of the `accelerate` method, increment the `x` component of the car's position by `0.01`.
3. Ensure that the car doesn't accelerate further if it is already going at the `maxSpeed`.
4. Inside of the car's `update` method, add the newly incremented `speed` value to current the `x` position.

Here's what the Speed indicator should read on the top left of the canvas:

```
Speed = 0
[Up arrow is pressed]
Speed = 0.01
Speed = 0.02
...
Speed = 5
Speed = 5
```

Reversing

Next, move the car towards the left side of the canvas when the user hits decelerate:

1. Call `car.decelerate()` when the user hits the `down` arrow key.
2. Inside of the `decelerate` method, decrement the `x` component of the car's position by `0.01`.
3. Ensure that the car doesn't decelerate further if it is already going at the (negative) `maxSpeed`.

Here's what the Speed indicator should read on the top left of the canvas:

```
Speed = 3
[Down arrow is pressed]
Speed = 2.99
Speed = 2.98
...
Speed = 0
Speed = -0.99
Speed = -0.98
...
Speed = -5
Speed = -5
```

Decelerating

Right now if the user accelerates the car and releases the `up` arrow key, the car keeps moving forward. Similarly, if the `down` arrow key is released, the car keeps moving in reverse. We want the car to slow down if it is in motion and no key presses are being registered. Think of this as taking your foot off of the gas pedal.

Friction

To achieve this deceleration effect, we must simulate `friction`. This will be a number that is multiplied to the current `speed` if the car is neither `accelerating` nor `reversing`.

1. Multiply the current `speed` by `friction` in the `decelerate` method but only if the car is in motion.
2. Check what happens to the `speed` indicator on the canvas when we `accelerate` or `reverse` and then let go of the corresponding key. Does it eventually reach zero?

- Go to the `displayStatus` method and remove the `.toFixed(2)` from the `speed` indicator.
 - Check what happens to the `speed` indicator on the canvas when we `accelerate` or `reverse` and then let go of the corresponding key. Does it eventually reach zero? Probably not! In actual fact, it's reaching a very very small value that is close, but not equal to, zero.
 - How can we fix this? Once `speed` reaches a super tiny value, go ahead and set it to `0`.
 - Go to the `displayStatus` method and put back the `.toFixed(2)` to the `speed` indicator so that the value displays nicely again.

Here's what the Speed indicator should read on the top left of the canvas:

```
Speed = 0
[Up arrow is pressed]
Speed = 0.01
Speed = 0.02
...
Speed = 5
[No keys are pressed]
Speed = 4.75
Speed = 4.51
Speed = 4.28
Speed = 4.07
...
Speed = 0
```

Steering

Our car only moves forward and back at the moment. This is not very exciting. We must now implement logic that can rotate the car if the user presses the `left` or `right` arrow keys.

⚠ Warning! This is the most math-intensive part of our car simulation!

Vectors

Right now our car only moves in the `x` direction. If we want it to move all around the canvas, then it also needs to move in the `y` direction. To do this, we must understand what a `vector` is. If you are new to vectors, or it's been a while since you've studied them, then please watch [this excellent Khan Academy video](#). There's also a great article for [analyzing vectors on the same website](#).

After understanding vectors and how we can break them down into their `x` and `y` components, we can finally start rotating our car:

- In the `turnLeft` method, increment `rotation` by `3`.
- In the `turnRight` method, decrement `rotation` by `3`.
- In the `update` method, put:

```
this.position.x += Math.cos(this.rotation * Math.PI / 180) * this.speed;  
this.position.y += Math.sin(this.rotation * Math.PI / 180) * this.speed;
```

Let's understand what's going on with the above code:

[5] Add the new x component to the current x

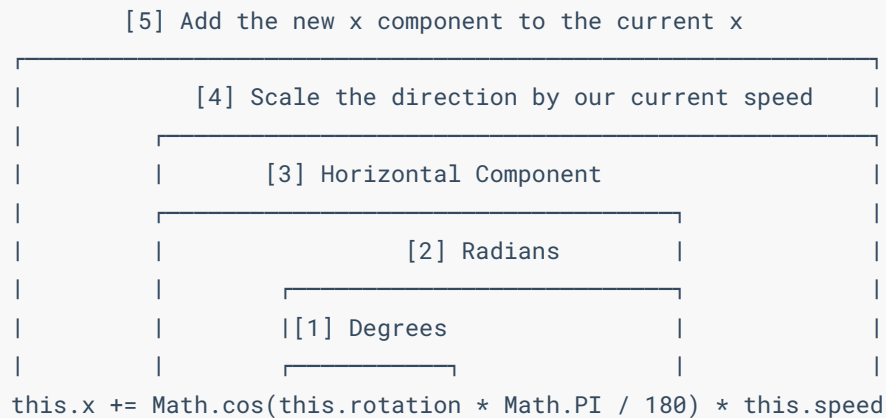
[4] Scale the direction by our current speed

[3] Horizontal Component

[2] Radians

[1] Degrees

```
this.x += Math.cos(this.rotation * Math.PI / 180) * this.speed
```



1. Get the current angle of the car (the vector in this case) in degrees.
2. Convert that angle to radians.
3. Calculate the `x` (or horizontal) component of the vector by taking the `cosine` of that angle.
4. Multiply the `x` component by the current speed to calculate where the car's `x` position should be based on this new rotation.
5. Add that to the current value of `x`.

Hopefully, after all that, you can now steer your car!

To achieve a more realistic effect, only allow turning when the car is in motion. Also, play around with the amount that the car gets rotated each time `turnRight` or `turnLeft` is called and find a value that you think is satisfying.

Maximizing Efficiency

There's one last thing we need to do before we're done with steering. Notice that if you were hold `up` and `right`, the rotation indicator keep going up and up. This is unnecessary and could cause an *overflow* error later if the number gets too big for the variable that it's in.

Once the car goes around once from `0` to `360` degrees and then keeps turning, it's not necessary to keep values that are greater than or equal to `360`. If you think about it, `360` degrees is the same as `0` degrees since in both cases, you're facing in the same direction. The same can be said for `361` and `1` degrees, `362` and `2` degrees, etc.

Try using the modulo `%` operator to limit the `rotation` value to be between `0` and `359` degrees at any given point.

Checking Bounds

We can now drive our car around anywhere we want on our canvas - *even right off of it* ! Try implementing logic that:

- 😊 **Easy**: Once the car goes out of bounds, reset its location to the other side of the canvas. For example, if the car goes off the *right* side of the canvas, reset its location to the *left* ; if the car goes off the *bottom* of the canvas, reset its location to the *top* , etc.
- 🤔 **Hard**: Checks if the car has hit the edge of the canvas, and if so, bounces off the "wall" 🚗💥🚗